



SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

ROHITH KANNA GV

128003214

A-44

SCHOOL OF COMPUTING

**SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH
ACADEMY**

**(SASTRA Deemed to be University)
Tirumalaisamudram, Thanjavur-613 401**

Exercise No. 10: Dining Philosopher's problem

Write a program to simulate Dining Philosopher's problem

Objectives

To design a solution that ensures the philosophers can safely share the forks and avoid deadlock or starvation (goes hungry).

Theory or Concept

Each philosopher must alternate between thinking and eating. Eating is possible only when two forks are available. It is used to handle synchronization and concurrency in process execution related to resource sharing and mutual exclusion.

Procedure / Algorithm

Step1: Represent each chopstick with a semaphore.

Step2: A philosopher tries to grab a chopstick by executing a wait () operation on that semaphore.

Step3: A philosopher releases her chopsticks by executing the signal () operation on the appropriate semaphores.

When each philosopher tries to grab her right chopstick, philosopher will be delayed forever (deadlock) to handle the situation allow a philosopher to pick up her chopsticks only if both chopsticks are available (the philosopher must pick them up in a critical section).

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#define NUM_PHILOSOPHERS 5
sem_t forks[NUM_PHILOSOPHERS];
sem_t mutex;
void take_fork(int id) {
    sem_wait(&mutex);
    sem_wait(&forks[(id + 1) % NUM_PHILOSOPHERS]);
    sem_wait(&forks[id]);
    sem_post(&mutex);
}
void put_fork(int id) {
    sem_post(&forks[id]);
    sem_post(&forks[(id + 1) % NUM_PHILOSOPHERS]);
}
void test_blocked_philosophers() {
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
```

```

if (sem_trywait(&forks[i]) == 0) {
printf("Philosopher %d is not blocked.\n", i);
sem_post(&forks[i]);
} else {
printf("Philosopher %d is blocked.\n", i);
}
}
printf("\n\n");
}
int main() {
int ch;
sem_init(&mutex, 0, 1);
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
sem_init(&forks[i], 0, 1);
}
printf("Initial state:\n");
test_blocked_philosophers();
printf("\n");
printf("With deadlock in the execution (1/0): ");
scanf("%d",&ch);
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
if(ch==0) {
printf("Philosopher %d wants to take forks:\n", i);
take_fork(i);
sleep(1);
printf("Philosopher %d took forks.\n\n", i);
printf("... State after taking forks ...\n");
test_blocked_philosophers();
sleep(1);
printf("\n");
printf("Philosopher %d releases forks:\n", i);
put_fork(i);
sleep(1);
printf("Philosopher %d released forks.\n\n", i);
printf("... State after releasing forks ...\n");
test_blocked_philosophers();
sleep(1);
printf("\n");
}
else
{
printf("Philosopher %d wants to take forks:\n", i);
take_fork(i);
sleep(1);
printf("Philosopher %d took forks.\n\n", i);
printf("... State after taking forks ...\n");
test_blocked_philosophers();

```

```

sleep(1);
printf("\n");
int next_phil = (i + 1) % NUM_PHILOSOPHERS;
printf("Philosopher %d wants to take forks:\n", next_phil);
take_fork(next_phil);
sleep(1);
printf("Philosopher %d took forks.\n\n", next_phil);
printf("... State after taking forks ...\n");
test_blocked_philosophers();
sleep(1);
printf("\n");
}
}
sem_destroy(&mutex);
for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
sem_destroy(&forks[i]);
}
return 0;
}

```

Output

Without deadlock:

```

Activities Terminal
Thu 09:53
guest-4cp7kb@admin25-VirtualBox: ~
collectd: error: id returned i exit status
guest-4cp7kb@admin25-VirtualBox: ~$ gcc -o DP DP.c -pthread
guest-4cp7kb@admin25-VirtualBox: ~$ ./DP
Initial state:
Philosopher 0 is not blocked.
Philosopher 1 is not blocked.
Philosopher 2 is not blocked.
Philosopher 3 is not blocked.
Philosopher 4 is not blocked.

With deadlock in the execution (1/0): 0
Philosopher 0 wants to take forks.
Philosopher 0 took forks.

... State after taking forks ...
Philosopher 0 is blocked.
Philosopher 1 is blocked.
Philosopher 2 is not blocked.
Philosopher 3 is not blocked.
Philosopher 4 is not blocked.

Philosopher 0 releases forks.
Philosopher 0 released forks.

... State after releasing forks ...
Philosopher 0 is not blocked.
Philosopher 1 is not blocked.
Philosopher 2 is not blocked.
Philosopher 3 is not blocked.
Philosopher 4 is not blocked.

*** Philosopher 1 wants to take forks:
*** Philosopher 1 took forks.

```

```
Activities Terminal
Thu 09:33
guest-8cp7kb@admin@25-VirtualBox: -

File Edit View Search Terminal Help
Philosopher 1 wants to take forks:
Philosopher 1 took forks.

... State after taking forks ...
Philosopher 0 is not blocked.
Philosopher 1 is blocked.
Philosopher 2 is blocked.
Philosopher 3 is not blocked.
Philosopher 4 is not blocked.

Philosopher 1 releases forks:
Philosopher 1 released forks.

... State after releasing forks ...
Philosopher 0 is not blocked.
Philosopher 1 is not blocked.
Philosopher 2 is not blocked.
Philosopher 3 is not blocked.
Philosopher 4 is not blocked.

Philosopher 2 wants to take forks:
Philosopher 2 took forks.

... State after taking forks ...
Philosopher 0 is not blocked.
Philosopher 1 is not blocked.
Philosopher 2 is blocked.
Philosopher 3 is blocked.
Philosopher 4 is not blocked.

Philosopher 2 releases forks:
Philosopher 2 released forks.
```

```
Activities Terminal
Thu 09:34
guest-8cp7kb@admin@25-VirtualBox: -

File Edit View Search Terminal Help
Philosopher 2 releases forks:
Philosopher 2 released forks.

... State after releasing forks ...
Philosopher 0 is not blocked.
Philosopher 1 is not blocked.
Philosopher 2 is not blocked.
Philosopher 3 is not blocked.
Philosopher 4 is not blocked.

Philosopher 3 wants to take forks:
Philosopher 3 took forks.

... State after taking forks ...
Philosopher 0 is not blocked.
Philosopher 1 is not blocked.
Philosopher 2 is not blocked.
Philosopher 3 is blocked.
Philosopher 4 is blocked.

Philosopher 3 releases forks:
Philosopher 3 released forks.

... State after releasing forks ...
Philosopher 0 is not blocked.
Philosopher 1 is not blocked.
Philosopher 2 is not blocked.
Philosopher 3 is not blocked.
Philosopher 4 is not blocked.

Philosopher 4 wants to take forks:
Philosopher 4 took forks.
```

```
Activities Terminal Thu 09:35
guest-9cp7kb@adminin25-VirtualBox: -
File Edit View Search Terminal Help

#Philosopher 3 releases forks:
#Philosopher 3 released forks.

... State after releasing forks ...
#Philosopher 0 is not blocked.
#Philosopher 1 is not blocked.
#Philosopher 2 is not blocked.
#Philosopher 3 is not blocked.
#Philosopher 4 is not blocked.

#Philosopher 4 wants to take forks:
#Philosopher 4 took forks.

... State after taking forks ...
#Philosopher 0 is blocked.
#Philosopher 1 is not blocked.
#Philosopher 2 is not blocked.
#Philosopher 3 is not blocked.
#Philosopher 4 is blocked.

#Philosopher 4 releases forks:
#Philosopher 4 released forks.

... State after releasing forks ...
#Philosopher 0 is not blocked.
#Philosopher 1 is not blocked.
#Philosopher 2 is not blocked.
#Philosopher 3 is not blocked.
#Philosopher 4 is not blocked.

guest-9cp7kb@adminin25-VirtualBox:~$
```

With deadlock:

```
Activities Terminal Thu 10:00
guest-9cp7kb@adminin25-VirtualBox: -
File Edit View Search Terminal Help

guest-9cp7kb@adminin25-VirtualBox:~$ ./DP
Initial state:
#Philosopher 0 is not blocked.
#Philosopher 1 is not blocked.
#Philosopher 2 is not blocked.
#Philosopher 3 is not blocked.
#Philosopher 4 is not blocked.

With deadlock in the execution (1/0): 1
#Philosopher 0 wants to take forks:
#Philosopher 0 took forks.

... State after taking forks ...
#Philosopher 0 is blocked.
#Philosopher 1 is blocked.
#Philosopher 2 is not blocked.
#Philosopher 3 is not blocked.
#Philosopher 4 is not blocked.

#Philosopher 1 wants to take forks:

```

Observation

Write the pseudo code for dining philosopher problem

The simulation uses a global `mutex` semaphore to ensure a philosopher picks up forks only when **both** left and right resources are simultaneously available, effectively eliminating the circular-wait condition that triggers a deadlock.

Pseudo-code for Dining Philosopher Problem

text

```
// Initialization

semaphore fork[5] = {1, 1, 1, 1, 1};

semaphore mutex = 1;


// Philosopher i process structure

while (true) {

    // Thinking state

    think();


    // Critical Section to grab both forks safely

    wait(mutex);

    wait(fork[i]);           // Grab left fork

    wait(fork[(i + 1) % 5]); // Grab right fork

    signal(mutex);

    // Eating state

    eat();


    // Release both forks

    signal(fork[i]);

    signal(fork[(i + 1) % 5]);

}
```

RESULT:

Thus, the **Dining Philosopher's problem** was successfully simulated using POSIX semaphores to demonstrate deadlock prevention and resource contention.